Team Reference Guide: Smart Multi-Canteen Pre-Order Management System (QuickBite)

1. Project Foundational Overview

The QuickBite system is the official digital solution for SLIIT Kandy Uni to eliminate dining congestion. This platform enables students and lecturers to pre-order meals from various campus canteens, ensuring food is ready upon arrival. The system architecture supports three distinct user tiers, each with specific access privileges and operational requirements.**Primary Goal:** Drastic reduction of queue times through a centralized, mobile-first pre-ordering and management ecosystem.

User Role Action Summary

User Role,Primary Responsibilities & Actions
User (Student/Lecturer),"View canteens, browse menu items, place pre-orders, process payments, submit feedback/ratings, and view active promotions."
Owner (Canteen Staff),"Manage canteen details, update food item listings, process incoming orders, reply to user complaints , and create/manage promotions."
Admin (System Manager),"Monitor all canteen activities, manage user and owner accounts, generate and view system-wide reports , and monitor payment activities."

2. SLIIT Academic Compliance & Marking Criteria

This project is the mandatory Group Assignment for **SE2020 – Web and Mobile Technologies (Year 2, Semester 2, 2026)** . Failure to adhere to these administrative rules will result in immediate disqualification.

Mandatory Academic Rules

- **Group Configuration:** Strictly 6 members per group.
- **Weightage:** 20% of the total module grade (100 marks scaled).
- **Duration:** 8-week development cycle.
- **Architectural Constraint: Firebase-only projects are strictly prohibited.** The system must be built using a Node.js + MongoDB backend.
- **Deployment Policy:** Localhost-only demos are not permitted for final evaluation. The system must be demonstrated via a live, hosted environment.

Individual Mark Sheet Breakdown

Component,Marks,Assessment Method
Code / Technical Implementation,40 Marks,"Git activity, system testing, and evidence of individual module ownership."
Individual Viva,60 Marks,Individual questioning on technical logic and module contributions.

Viva Evaluation Criteria

Every member must attend the Viva. **Note:** Individual marks will be significantly reduced if a student fails to explain the logic behind their specific module.

1. **Explaining Own Module in Detail (20 Marks):** Clear explanation of logic, functions, and data flow.
2. **Integration with Other Modules (10 Marks):** Understanding system-wide connectivity.
3. **Backend & Database Concepts (10 Marks):** Knowledge of schemas, routes, and controllers.
4. **Mobile & API Interaction Logic (10 Marks):** Understanding request flow and UI behavior.
5. **Problem Solving & Debugging (10 Marks):** Handling "what if" scenarios and live debugging.

3. Technology Stack & Deployment Architecture

Core Technologies

- **Frontend:** React Native / Expo
- **Backend:** Node.js / Express.js
- **Database:** MongoDB (Atlas for cloud hosting)

Required Libraries & Standards

- **Axios:** Standard for all API communication.
- **React Navigation:** Mandatory for screen management.
- **Expo Image Picker:** The project standard for handling file uploads.
- **Async Storage:** For local data persistence and session management.

Deployment & Hosting Requirements

The mobile application **must** connect to a hosted API. Developers must configure environment variables (.env) for all sensitive credentials.

- **Approved Platforms:** AWS, Render, Railway, or DigitalOcean.
- **Requirement:** Live API endpoints must be fully functional during the final demo.

4. 6-Member Workload & Technical Specifications

Each member is responsible for the Full Stack implementation (Backend CRUD, API routes, Controllers, and Mobile UI) of their assigned entity.

1. Canteen Management

- **Responsibilities:** Full CRUD for canteen entities, image/logo uploads, and hour management.
- **Mandatory Fields:** canteenName, location, contactDetails, ownerDetails, openingTime, closingTime, canteenImage.

2. Food Item Management

- **Responsibilities:** Price/quantity management, availability toggles, and category filtering.
- **Mandatory Fields:** foodItemName, category, price, availableQuantity, description, foodImage, canteenID.

3. Order Management

- **Responsibilities:** Cart logic, pre-order placement, **order history** , and the **pickup process** .
- **Status Workflow:** pending, confirmed, preparing, ready, completed, cancelled.
- **Mandatory Fields:** userID, canteenID, orderedItems, totalPrice, pickupTime, status, specialNotes, requestImage (optional).

4. Payment Management

- **Responsibilities:** Transaction tracking, payment proof uploads, and status monitoring.
- **Status Workflow:** pending, paid, failed, refunded.
- **Mandatory Fields:** orderID, paymentMethod, amount, paymentDate, paymentStatus, transactionReference, paymentProofImage.

5. Feedback Management

- **Responsibilities:** Ratings, complaints with image evidence, and **owner/admin response logic** .
- **Mandatory Fields:** userID, canteenID, orderID, rating, feedbackMessage, complaintType, complaintImage, response, status.

6. Promotions & Discounts Management

- **Responsibilities:** Creation of meal deals, banner uploads, and active/inactive state scheduling.
- **Mandatory Fields:** promotionTitle, description, bannerImage, discountPercentage, startDate, endDate, targetItem/Canteen, activeStatus.

Group Responsibility: Authentication & Security

The entire group must implement a unified security layer:

- **Auth:** User Registration and Login APIs for Users, Owners, and Admins.
- **Security:** Password encryption/hashing (Bcrypt) and JWT-based session management for protected routes.

5. Backend & Frontend Structural Standards

Backend Architecture

The backend must follow a strict RESTful pattern with the following mandatory directory structure:

- /models: MongoDB Schemas.

- /controllers: Business logic and CRUD operations.
- /routes: API endpoint definitions.
- /middleware: Auth verification and file upload handling.
- /config: Database and environment configurations.
- **Requirement:** All API responses must utilize appropriate HTTP status codes (200, 201, 400, 401, 404, 500).

Mobile Frontend Requirements

- **UI/UX:** Must be mobile-friendly, fast, and visually appealing. No hardcoded data is permitted.
- **Navigation Flow:**
- **Core:** Home → Promotions, Home → Feedback, Home → Profile.
- **User Screens:** Canteen List, Food Item, Cart, Order, Payment, Feedback, Promotion, Profile.
- **Owner/Admin Screens:** Canteen Dashboard, Food Management, Order Processing, Promotion Management, Feedback Review.

6. Shared System Features

- **File Upload Support:** Integrated across all modules via a standardized pipeline (e.g., Multer) for canteen logos, food images, payment receipts, complaint evidence, and promotional banners.
- **Search and Filter Features:**
- Global search for canteens and food items.
- Filtering by category, canteen, or order status.
- **Promotions Filter:** Must filter by specific date range or active state.
- **Role-Based Access Control (RBAC):** Strict enforcement of endpoint access based on user tier (User vs. Owner vs. Admin).

7. GitHub Collaboration Workflow

📌 Core Rule

NEVER push directly to the 'main' branch. Direct pushes bypass the review process and risk system instability.

Branch Naming Convention

All members must use the following format: feature/module-name

- *Example:* feature/payment-management or feature/canteen-crud.

Daily Routine

1. **Step 0 (Pull Before Coding):** Always pull the latest changes before starting work.
2. **Step 1:** Switch to main and pull the latest code.
3. **Step 2:** Switch to your functional branch.
4. **Step 3:** Merge main into your branch to resolve conflicts locally.

5. **Step 4:** Perform coding, validation, and testing.
6. **Step 5:** Add, commit, and push. **Commit messages must be clear.** (e.g., "Add: Canteen image upload controller" vs "Update").

8. Final Documentation & Viva Preparation

Mandatory Final Report Components

1. **Problem Statement:** Detailed context on queue reduction.
2. **System Architecture Diagram:** High-level tech stack and flow.
3. **Database Schema Diagram:** Visual representation of all collections and relationships.
4. **API Endpoint Table:** Documentation of all routes, methods, and expected payloads.
5. **Team Responsibility Breakdown:** Clear mapping of members to modules.

Individual Viva Focus Checklist

Members must be prepared to demonstrate and explain:

- **Controller Logic:** The specific business rules of your module.
- **Route Design:** Structure and security of your API endpoints.
- **Data Validation:** How you prevent invalid data from entering the database.
- **State Management:** Handling UI states and data flow in the mobile app.
- **Error Handling:** Displaying meaningful error messages to the user.

Integrity Policies

- **Plagiarism:** A zero-tolerance policy is in effect. Plagiarism results in **zero marks** for the entire group.
- **AI Usage:** AI tools are for learning support only. Generating the full system via AI is prohibited. All code must be explainable by the author.